

■ Lekcja 02: Advanced SQL + performance mindset

CTE, window functions, **EXPLAIN QUERY PLAN** i pierwsze myślenie performance Data Engineera.

Cel lekcji:

Nie optymalizuję w ciemno.
Najpierw rozumiem wynik, potem grain, potem plan, potem koszt.

To nie jest lekcja tylko o przyspieszaniu query.

Performance jest soczewką, przez którą ćwiczysz advanced SQL:

- CTE jako narzędzie czytelności i review,
- window functions do running total i ranking,
- **EXPLAIN QUERY PLAN** jako evidence,
- myślenie o skali jak w rozmowie interview.

Problem

Query może być poprawne składniowo, ale nadal problematyczne:

- liczy złą metrykę,
- ma niejasny grain,
- czyta za dużo danych,
- robi kosztowny join,
- sortuje bez potrzeby,
- używa window function bez świadomego **ORDER BY**,
- wygląda profesjonalnie, ale nie ma evidence.

W tej lekcji uczysz się patrzeć na query jak Data Engineer: wynik, grain, CTE/window, plan, koszt i jedna sensowna poprawka.

Ćwiczenia nie są dodatkiem. To główna część lekcji.

Najważniejsza kolejność

correctness -> readability -> explainability -> cost awareness

To jest kolejność, nie lista równorzędnych opcji.

Nie można optymalizować query, które nie liczy dobrej metryki.

Ale też nie uczymy samej teorii performance. Każdy koncept ma ćwiczenie:

- **GROUP BY** i grain,
- CTE i review query,
- window function i running total,
- ranking Netflix-style,
- aggregate layer przy dużej skali.

SQL query to prośba do silnika danych:

```
przeczytaj dane,  
połącz tabele,  
odfiltruj rekordy,  
policz metrykę,  
posortuj wynik.
```

Performance zaczyna się od pytania:

```
Ile pracy każesz wykonać silnikowi?
```

Wskazówka: nie zaczynaj od słowa "indeks". Najpierw zapytaj, czy query liczy dobrą rzecz.

Przed tuningiem powiedz na głos

```
To query ma zwrócić ...  
Grain wyniku to ...  
Czyta tabele ...  
Filtruje po ...  
Potencjalnie drogie miejsce to ...  
Sprawdzę to przez EXPLAIN QUERY PLAN.
```

Szybkie query, które liczy złą metrykę, jest gorsze niż wolne query, które da się poprawić.

■ Skąd bierze się koszt query?

Koszt zwykle bierze się z pracy, którą musi wykonać silnik:

- liczba czytanych rekordów,
- liczba czytanych kolumn,
- joiny,
- sortowanie,
- agregacje,
- window functions,
- brak filtra po dacie lub partycji,
- niejasny grain.

Wskazówka: performance review to nie zgadywanie. Musisz pokazać evidence.

Query jest wolne, więc od razu dodaję indeks.

Lepszy wzorzec:

1. Czy wynik jest poprawny?
2. Jaki jest grain?
3. Ile danych czytam?
4. Co pokazuje EXPLAIN?
5. Jaka jedna zmiana zmniejsza koszt albo ryzyko?

EXPLAIN QUERY PLAN

EXPLAIN QUERY PLAN pokazuje, jak SQLite planuje wykonać query.

Na start szukasz odpowiedzi:

- z jakich tabel czytamy?
- czy widać **SCAN** czy **SEARCH**?
- czy pojawia się join?
- czy jest sort albo temp structure?
- czy query robi więcej pracy niż potrzeba?

 EXPLAIN nie jest magią

EXPLAIN nie mówi, czy wynik jest biznesowo poprawny.

EXPLAIN mówi, jaką pracę planuje wykonać silnik.

Dlatego przed EXPLAIN nadal musisz znać:

- cel query,
- grain wyniku,
- definicję metryki,
- klucze joinów,
- check walidacyjny.

Co to jest indeks?

Indeks to dodatkowa struktura danych obok tabeli.

Pomaga silnikowi szybciej znaleźć rekordy po konkretnej kolumnie.

W tej lekcji mamy np. indeks po `orders.status`:

```
CREATE INDEX idx_orders_status ON orders(status);
```

Jeśli query filtruje albo grupuje po `status`, SQLite może użyć indeksu zamiast czytać pełną tabelę w przypadkowej kolejności.

Bez indeksu:

silnik częściej musi przejrzeć wiele rekordów tabeli

Z indeksem:

silnik może szybciej znaleźć albo przejść rekordy według statusu

Indeks nie zmienia wyniku query. Zmienia możliwą drogę dojścia do wyniku.

Indeks: trade-off

Indeks nie jest darmowy.

Plus:

```
SELECT z filtrem albo joinem może czytać mniej danych.
```

Koszt:

```
INSERT/UPDATE/DELETE muszą aktualizować też indeks.  
Indeks zajmuje dodatkowe miejsce.  
Zły indeks może nie pomóc żadnemu query.
```

Dlatego nie zaczynasz od "dodaj indeks".

Zaczynasz od:

```
Jakie query jest wolne?  
Jaki filtr/join jest ważny?  
Co pokazuje EXPLAIN?
```

Co to jest covering index?

Zwykły indeks pomaga bazie znaleźć rekordy.

Ale czasem po znalezieniu rekordów baza musi jeszcze wrócić do pełnej tabeli po brakujące kolumny.

Covering index oznacza:

```
Indeks ma wszystko, czego query potrzebuje.  
Baza może odpowiedzieć z samego indeksu.  
Nie musi dodatkowo czytać pełnych rekordów z tabeli.
```

Przykład z tej lekcji:

```
SCAN orders USING COVERING INDEX idx_orders_status
```

To znaczy:

```
SQLite czyta indeks po statusie.  
Do policzenia COUNT(*) per status wystarczy mu ten indeks.  
Dlatego nie musi wracać do pełnej tabeli orders po inne kolumny.
```

SCAN table

silnik czyta wiele albo wszystkie rekordy z tabeli
performance: koszt rośnie razem z rozmiarem tabeli

SEARCH table USING INDEX

silnik potrafi szukać węzłej, np. przez indeks
performance: zwykle mniej czytania, jeśli filtr jest selektywny

SEARCH table USING COVERING INDEX

silnik czyta sam indeks, bo indeks ma kolumny potrzebne do wyniku
performance: często taniej niż czytanie indeksu i potem pełnej tabeli

JOIN

silnik dopasowuje rekordy z dwóch źródeł
w SQLite często widzisz to jako SCAN/SEARCH dla obu tabel
performance: koszt zależy od liczby rekordów przed joinem i indeksu na kluczu

USE TEMP B-TREE

silnik robi dodatkową strukturę, często dla ORDER BY albo GROUP BY
performance: sortowanie/grupowanie może zużyć pamięć albo dysk

EXPLAIN: jak to łączyć z performance?

Nie czytaj planu jako listy magicznych słów. Czytaj go jako listę prac, które musi zrobić silnik.

SCAN dużej tabeli

ryzyko: dużo odczytu; query może wolno rosnać ze skalą danych

SEARCH po indeksie

plus: silnik zawęży rekordy wcześniej

uwaga: indeks pomaga najbardziej, gdy filtr wybiera małą część danych

JOIN

ryzyko: jeśli przed joinem jest dużo rekordów, silnik ma dużo dopasowań do sprawdzenia

USE TEMP B-TREE

ryzyko: dodatkowe sortowanie albo grupowanie; koszt rośnie z liczbą rekordów

Pierwsze pytanie performance brzmi:

Ile rekordów wchodzi do tego kroku?

Drugie pytanie:

Czy mogę poprawnie biznesowo zmniejszyć dane wcześniej?

Przykład outputu z EXPLAIN

Przykład outputu:

```
SCAN orders USING COVERING INDEX idx_orders_status
```

Co to znaczy prosto:

```
SCAN orders USING COVERING INDEX idx_orders_status
```

SQLite czyta indeks po statusie. Indeks ma wystarczająco dużo danych,
żeby policzyć zamówienia per status bez czytania pełnej tabeli.

W innych query możesz zobaczyć też:

```
USE TEMP B-TREE FOR GROUP BY
```

To znaczy:

```
USE TEMP B-TREE FOR GROUP BY
```

SQLite tworzy tymczasową strukturę pomocniczą, żeby pogrupować rekordy.
To jest dodatkowa praca: baza musi zebrać podobne wartości razem i policzyć agregację.

Przykład 1: orders per status

Cel:

Policzyć liczbę zamówień per status.
Grain wyniku: jeden rekord = jeden status.

```
EXPLAIN QUERY PLAN
SELECT
  status,
  COUNT(*) AS orders_count
FROM orders
GROUP BY status;
```

Patrzysz na trzy rzeczy:

1. Jakie tabele są czytane?
2. Czy widać SCAN albo SEARCH?
3. Czy GROUP BY wymaga dodatkowej pracy?

Przykład 2: SELECT star vs jawne kolumny

Eksploracja:

```
EXPLAIN QUERY PLAN
SELECT *
FROM orders
WHERE order_date >= '2026-01-01'
      AND order_date < '2026-01-06';
```

Pipeline:

```
EXPLAIN QUERY PLAN
SELECT
  order_id,
  customer_id,
  order_date,
  status,
  channel
FROM orders
WHERE order_date >= '2026-01-01'
      AND order_date < '2026-01-06';
```

W SQLite plan może być podobny. W Parquet/warehouse jawne kolumny mogą oznaczać mniej odczytu.

Dlaczego?

SQLite zwykle trzyma rekord jako wiersz. Jeśli czytasz jeden rekord, baza i tak jest17 / 49 blisko całego wiersza.

SELECT star

SELECT * jest wygodne w eksploracji, ale słabe w pipeline.

Dlaczego:

- czyta niepotrzebne kolumny,
- ukrywa kontrakt danych,
- downstream może zepsuć się po zmianie schematu,
- w silnikach kolumnowych może oznaczać więcej odczytu.

Wskazówka: **SELECT *** jest OK, gdy pierwszy raz oglądasz dane. Finalne query powinno mieć jawne kolumny.

CTE jako narzędzie review

CTE nie jest automatyczną optymalizacją.

CTE pomaga wtedy, gdy nazwa kroku mówi, co już stało się z danymi.

CTE zaczyna się od **WITH** i tworzy nazwy, których możesz użyć później w głównym **SELECT**.

```
WITH paid_orders AS (...),  
     order_revenue AS (...)  
SELECT ...  
FROM order_revenue;
```

order_revenue nie jest tabelą z bazy. To nazwana część query dostępna tylko w tym jednym zapytaniu.

Dobre nazwy:

```
paid_orders  
order_revenue  
daily_revenue  
ranked_pages
```

Słabe nazwy:

```
tmp  
cte1  
data  
final2
```

CTE a kolejność wykonywania SQL

To się nie kłóci z kolejnością:

```
FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY
```

Ta kolejność mówi, jak czytać pojedynczy blok **SELECT**.

CTE dodaje krok wcześniej: najpierw deklarujesz nazwy pomocnicze przez **WITH**, potem główny **SELECT** może ich użyć w **FROM**.

Czytaj przykład 3 tak:

1. **WITH** paid_orders AS (...)
nazwa paid_orders jest dostępna dalej
2. **WITH** order_revenue AS (... FROM paid_orders ...)
nazwa order_revenue jest dostępna dalej
3. **SELECT** ... FROM order_revenue
główny **SELECT** bierze dane z nazwanej części query

W każdym z tych **SELECT** nadal obowiązuje logika: najpierw **FROM**, potem **WHERE**, potem **GROUP BY**, potem lista **SELECT**.

 Czy baza naprawdę wykonuje CTE po kolei?

Nie zawsze.

To ważne rozróżnienie:

Jak czytam query jako człowiek:

WITH krok 1 -> WITH krok 2 -> finalny SELECT

Jak silnik może to wykonać:

optimizer może przepisać, połączyć albo wstawić CTE do większego planu

Dla nauki SQL traktuj CTE jako nazwane kroki myślenia.

Dla performance pamiętaj: CTE nie gwarantuje automatycznie, że baza fizycznie zapisze wynik pośredni albo wykona wszystko dokładnie w tej kolejności.

Przykład 3: revenue per channel przez CTE

Cel:

Policzyć przychód per channel tylko dla paid orders.
Grain wyniku: jeden rekord = jeden channel.

```
WITH paid_orders AS (  
    SELECT  
        order_id,  
        channel  
    FROM orders  
    WHERE status = 'paid'  
),  
order_revenue AS (  
    SELECT  
        po.order_id,  
        po.channel,  
        SUM(oi.quantity * oi.unit_price) AS order_revenue  
    FROM paid_orders AS po  
    JOIN order_items AS oi  
        ON po.order_id = oi.order_id  
    GROUP BY po.order_id, po.channel  
)  
SELECT  
    channel,  
    SUM(order_revenue) AS total_revenue  
FROM order_revenue  
GROUP BY channel;
```

 Dlaczego ten CTE ma sens?

paid_orders:

Grain: jeden rekord = jedno opłacone zamówienie.
Filtr: status = 'paid'.
Ryzyko: jeśli filtr jest zły, cała metryka revenue jest zła.

order_revenue:

Grain: jeden rekord = jedno zamówienie.
Metryka: SUM(quantity * unit_price).
Ryzyko: join orders -> order_items zmienia grain na pozycje zamówienia, więc agregujemy z powrotem do order_id.

Finalny SELECT:

Grain: jeden rekord = jeden channel.

Window functions: po co?

Używasz window function, gdy chcesz policzyć metrykę, ale nie chcesz zgubić aktualnych wierszy.

Najważniejszy kontrast:

```
GROUP BY      = zwiija wiersze  
window function = zostawia wiersze i dodaje kolumnę
```

Przykład:

```
daily_revenue ma 5 dni.  
Po running total nadal ma 5 dni.  
Dochodzi tylko kolumna running_revenue.
```

Window functions: kiedy?

Najczęstsze użycia:

- running total,
- ranking,
- deduplikacja najnowszego rekordu,
- porównanie z poprzednim dniem,
- moving average.

`RANK()`, `ROW NUMBER()` i `DENSE RANK()` to też window functions: zostawiają wiersze i dopisują numer zależny od `ORDER BY` oraz decyzji, co zrobić z remisami.

Pytanie kontrolne:

Czy chcę zmienić liczbę wierszy?

Jeśli nie, window function często jest dobrym kandydatem.

Window functions: najważniejsze rodziny

Nie musisz znać wszystkich funkcji naraz. Ważne jest rozpoznanie rodzin:

```
Aggregate windows: SUM, AVG, COUNT, MIN, MAX
Ranking windows:   ROW_NUMBER, RANK, DENSE_RANK, NTILE
Offset windows:    LAG, LEAD
Value windows:     FIRST_VALUE, LAST_VALUE, NTH_VALUE
Distribution:      PERCENT_RANK, CUME_DIST
```

W tej lekcji wymagamy głównie `SUM(...) OVER (...)`, `RANK()` i rozróżnienia `RANK()` vs `ROW_NUMBER()`.

Najwięcej ćwiczymy funkcje rankingowe, bo często wracają w zadaniach interview i w analitycznym SQL.

Window functions: jak czytać?

Wzorzec:

```
SUM(daily_revenue) OVER (  
    ORDER BY order_date  
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
) AS running_revenue
```

Czytaj od zewnątrz:

```
SUM(daily_revenue) -> co liczę  
OVER (...)         -> po których wierszach liczę  
ORDER BY           -> w jakiej kolejności  
ROWS BETWEEN       -> od którego do którego wiersza
```

 Window: co funkcja widzi?

Window to wiersze widoczne dla aktualnego wiersza.

Dla running total:

```
2026-01-01 widzi: 2026-01-01
2026-01-02 widzi: 2026-01-01, 2026-01-02
2026-01-03 widzi: 2026-01-01, 2026-01-02, 2026-01-03
```

Dlatego suma narasta.

 ORDER BY: po co?

Running total oznacza: narastająco w konkretnej kolejności.

Bez kolejności nie wiadomo, co jest wcześniej i później.

```
SUM(daily_revenue) OVER ()
```

To nie jest running total. To zwykle suma globalna powtórzona przy każdym wierszu.

Do running total potrzebujesz:

```
SUM(daily_revenue) OVER (ORDER BY order_date)
```

 Frame: po co ta długa linia?

```
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

Po polsku:

```
Dla aktualnego dnia weź wszystkie dni od początku do teraz.
```

Elementy:

ROWS	-> licz wiersze po sortowaniu
UNBOUNDED PRECEDING	-> zacznij od pierwszego wiersza
CURRENT ROW	-> skończ na aktualnym wierszu

 PARTITION BY: kiedy?

PARTITION BY oznacza: zrób osobne okno dla każdej grupy.

Bez partition:

```
jeden running total dla całej tabeli
```

Z partition:

```
osobny running total per channel
```

```
SUM(daily_revenue) OVER (  
  PARTITION BY channel  
  ORDER BY order_date  
) AS channel_running_revenue
```

Najczęstszy błąd

Window function musi działać na dobrym grainie.

Do running total po dniach najpierw potrzebujesz:

```
daily_revenue  
jeden rekord = jeden dzień
```

Jeśli policzysz running total na `order items`, wynik będzie narastał po pozycjach zamówienia, a nie po dniach.

- | | |
|--------------------------------------|------------------------|
| 1. Jaki grain ma wejście? | |
| 2. Czy potrzebuję osobnych grup? | -> PARTITION BY |
| 3. Czy wynik zależy od kolejności? | -> ORDER BY |
| 4. Od kiedy do kiedy funkcja patrzy? | -> frame |
| 5. Jaką metrykę liczę? | -> SUM/RANK/ROW_NUMBER |

Minimalny running total:

```
WITH daily_revenue AS (  
    SELECT  
        order_date,  
        SUM(line_revenue) AS daily_revenue  
    FROM source_rows  
    GROUP BY order_date  
)  
SELECT  
    order_date,  
    daily_revenue,  
    SUM(daily_revenue) OVER (  
        ORDER BY order_date  
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
    ) AS running_revenue  
FROM daily_revenue;
```

GROUP BY czy window function?

Chcę zwinąć wiersze do jednej grupy
-> GROUP BY

Chcę zachować wiersze i dodać metrykę
-> window function

Chcę running total po dacie
-> SUM(...) OVER (ORDER BY order_date)

Pytanie kontrolne:

Czy running total ma sens bez ORDER BY order_date?

Przykład 4: running total

Cel:

Pokazać daily revenue i narastający revenue po dacie.
Grain wyniku: jeden rekord = jeden dzień.

```
WITH daily_revenue AS (  
    SELECT  
        o.order_date,  
        SUM(oi.quantity * oi.unit_price) AS daily_revenue  
    FROM orders AS o  
    JOIN order_items AS oi  
        ON o.order_id = oi.order_id  
    WHERE o.status = 'paid'  
    GROUP BY o.order_date  
)  
SELECT  
    order_date,  
    daily_revenue,  
    SUM(daily_revenue) OVER (  
        ORDER BY order_date  
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
    ) AS running_revenue  
FROM daily_revenue  
ORDER BY order_date;
```

■ Jak działa running total?

Dla każdego dnia window function patrzy na uporządkowane wiersze:

```
2026-01-01 -> suma od początku do 2026-01-01  
2026-01-02 -> suma od początku do 2026-01-02  
2026-01-03 -> suma od początku do 2026-01-03
```

ORDER BY order_date jest częścią logiki biznesowej.

Bez **ORDER BY** nie ma jasnego pojęcia "narastająco".

Przykład 5: ranking jak w Netflix

Pytanie interview:

```
Rank titles by total view hours during the 8-day period ending on
2024-02-21.
```

Minimalny wzorzec:

```
SELECT
    title_id,
    title_desc,
    genre,
    total_view_hours,
    RANK() OVER (ORDER BY total_view_hours DESC) AS title_rank
FROM title_views_8d
ORDER BY title_rank;
```

Wskazówka: **RANK()** zachowuje remisy. **ROW NUMBER()** zawsze nada unikalny numer, nawet gdy dwa tytuły mają ten sam wynik.

To jest decyzja biznesowa: czy dwa identyczne wyniki mają mieć tę samą pozycję, czy potrzebujesz sztucznie wybrać jeden rekord.

Netflix-style: najpierw agregacja

Przy małej skali możesz czytać fakt bezpośrednio.

Przy dużej skali pytasz:

```
Czy tabela ma 1M, 1B czy 10B wierszy dziennie?  
Czy data jest partition key?  
Czy istnieje aggregate layer?
```

Jeżeli `streaming_session` ma 1B wierszy dziennie, to ranking za 8 dni może skanować 8B wierszy.

Lepszy produkcyjny wzorzec:

```
streaming_session -> title_daily_views -> ranking query
```

Przykład 6: aggregate layer

Tabela dzienna:

```
INSERT INTO title_daily_views
SELECT
    title_id,
    stream_start_utc_date,
    SUM(total_view_sec) AS total_view_sec
FROM streaming_session
WHERE stream_start_utc_date = DATE '2024-02-21'
GROUP BY title_id, stream_start_utc_date;
```

Ranking czyta agregat:

```
SELECT
    title_id,
    SUM(total_view_sec) / 3600.0 AS total_view_hours,
    RANK() OVER (ORDER BY SUM(total_view_sec) DESC) AS title_rank
FROM title_daily_views
WHERE stream_start_utc_date BETWEEN DATE '2024-02-14' AND DATE '
2024-02-21'
GROUP BY title_id;
```

Co powiedzieć na rozmowie

Filtruję po kolumnie partycji, więc silnik czyta tylko potrzebne dni.

Przy 1B wierszy dziennie nie chcę za każdym razem skanować faktu.
Buduję albo wykorzystuję aggregate layer.

RANK() wybieram, gdy remisy mają mieć ten sam ranking.
ROW_NUMBER() wybieram, gdy potrzebuję dokładnie jednego rekordu.

Weighted average: pytanie interview

Pytanie interview:

Calculate average price per night for each country in 2020.

Antywzorzec:

```
AVG(total_price_usd / stay_length)
```

Problem: rezerwacja 1-dniowa i 30-dniowa mają taką samą wagę.

Lepsza metryka:

```
SUM(total_price_usd) / NULLIF(SUM(stay_length), 0)
```

To jest weighted average: każda noc ma taką samą wagę, a nie każda rezerwacja.

Wskazówka: performance nie pomoże, jeśli metryka jest źle zdefiniowana.

Weighted average: pytanie doprecyzowujące

Przed SQL-em zapytaj:

```
Czy "w 2020" oznacza rezerwacje w pełni zawarte w 2020,  
czy rezerwacje nachodzące na 2020?
```

To zmienia query.

Wariant prosty:

```
WHERE reservation_start_date >= DATE '2020-01-01'  
      AND reservation_end_date < DATE '2021-01-01'
```

Wariant overlap wymaga policzenia `nights_in 2020`.

Partitioning vs clustering

Tego nie implementujemy w SQLite. Budujemy intuicję na później: DuckDB, Parquet, warehouse i lakehouse.

Partitioning = dzieli dane na duże części, często po dacie.
Clustering = układa podobne rekordy blisko siebie wewnątrz danych.

Wskazówka: gdy dane są duże, filtr po dacie może pozwolić silnikowi pominąć całe kawałki danych zamiast czytać wszystko.

 Wzorzec 1: jedna zmiana naraz

Antywzorzec:

Jednocześnie dodaję filtr, zmieniam join, usuwam kolumny i przepisuję CTE.

Problem:

Nie wiem, która zmiana pomogła.

Lepszy wzorzec:

baseline -> EXPLAIN -> jedna zmiana -> EXPLAIN -> notatka

Wzorzec 2: filtruj możliwie wcześnie

Antywzorzec:

```
WITH all_joined AS (  
    SELECT *  
    FROM orders AS o  
    JOIN order_items AS oi ON o.order_id = oi.order_id  
)  
SELECT *  
FROM all_joined  
WHERE status = 'paid';
```

Problem: najpierw łączysz więcej danych, potem dopiero odrzucasz część rekordów.

Lepszy wzorec: filtr przed joinem

```
WITH paid_orders AS (  
    SELECT  
        order_id,  
        customer_id,  
        order_date,  
        channel  
    FROM orders  
    WHERE status = 'paid'  
)  
SELECT  
    po.order_id,  
    po.customer_id,  
    oi.product_id,  
    oi.quantity * oi.unit_price AS line_revenue  
FROM paid_orders AS po  
JOIN order_items AS oi  
    ON po.order_id = oi.order_id;
```

Wskazówka: filtr musi być zgodny z definicją biznesową. `status = 'paid'` ma sens, jeśli liczysz paid revenue.

Wzorzec 3: wyjaśnij query po polsku

Przed oddaniem query napisz:

```
Cel:  
Tabele wejściowe:  
Grain wyniku:  
Filtry:  
Klucze joinów:  
Kosztowne operacje:  
Check walidacyjny:
```

Wskazówka: jeżeli nie umiesz opisać query prostymi zdaniami, prawdopodobnie nie rozumiesz jeszcze grain, filtra albo kosztownej operacji.

Mini-checklist

Przed uznaniem query za gotowe:

- Czy wynik jest biznesowo poprawny?
- Czy znasz grain wyniku?
- Czy wybierasz jawne kolumny?
- Czy filtr statusu/daty jest możliwie wcześnie?
- Czy `EXPLAIN QUERY PLAN` został uruchomiony?
- Czy opisałeś jedno potencjalnie drogie miejsce?
- Czy poprawka była jedna naraz?

Closing check

Powiedz na głos:

```
Nie optymalizuję w ciemno.  
Najpierw sprawdzam poprawność i grain.  
Potem czytam EXPLAIN.  
Potem zmieniam jedną rzecz i porównuję plan albo wynik.
```